

Design and Comparison of 4×4 Binary Array and Booth Multipliers

Project Objective

- Design a **4×4 binary array multiplier** using Verilog RTL.
- Design a **4×4 Booth multiplier** for signed multiplication.
- Verify both designs using a **common testbench**.
- Analyze **simulation waveforms** and compare the architectures.
- Perform **synthesis analysis** if the tool flow is available.

4×4 BINARY MULTIPLICATION

- Binary multiplication is performed using **partial products**.
- Each partial product is generated using **AND operations**.
- Partial products are shifted according to the **multiplier bit position**.
- The shifted partial products are added to obtain the **8-bit product**.
- The implemented array multiplier is **unsigned**.

Array Multiplier Architecture

- Inputs: **A[3:0]** and **B[3:0]**
- Generate four partial products:
 - **pp0**
 - **pp1**
 - **pp2**
 - **pp3**
- Shift each partial product by **0, 1, 2 and 3 positions**.
- Add all shifted partial products.
- Output: **Product[7:0]**

BOOTH MULTIPLICATION

Booth Multiplication

- Booth multiplication reduces **repeated addition operations**.
- It examines the current multiplier bit and the previous bit (**Q-1**).
- **01** → **A = A + M**
- **10** → **A = A - M**
- **00 or 11** → **No arithmetic operation**

- After each operation, an **arithmetic right shift** is performed.

Booth Multiplier Architecture

- **M**: Signed multiplicand
- **Q**: Signed multiplier
- **A**: Accumulator
- **Q-1**: Previous multiplier bit
- The algorithm is repeated **4 times** for a 4×4 multiplier.
- Final **A and Q bits** form the 8-bit signed product.

VERILOG RTL IMPLEMENTATION

Verilog RTL Implementation

- Array multiplier uses **continuous assignments** for partial products and addition.
- Booth multiplier uses an **always block**, registers, conditional add/subtract and shifting.
- Both designs are **combinational implementations**.
- The RTL is written to be **synthesizable**.
- Separate modules make **verification and comparison easier**.

4×4 Binary Multiplication:

```
module array_multiplier_4x4 (
    input [3:0] A,
    input [3:0] B,
    output [7:0] Product
);

    wire [3:0] pp0;
    wire [3:0] pp1;
    wire [3:0] pp2;
    wire [3:0] pp3;

    // Generate partial products
    assign pp0 = A & {4{B[0]}};
    assign pp1 = A & {4{B[1]}};
    assign pp2 = A & {4{B[2]}};
    assign pp3 = A & {4{B[3]}};

    // Add shifted partial products
    assign Product = {4'b0000, pp0}
        + {3'b000, pp1, 1'b0}
        + {2'b00, pp2, 2'b00}
        + {1'b0, pp3, 3'b000};
```

Endmodule

Booth Multiplication

```
module booth_multiplier_4x4 (  
    input signed [3:0] M,  
    input signed [3:0] Q,  
    output reg signed [7:0] Product  
);  
  
    reg signed [4:0] A;  
    reg signed [4:0] M_ext;  
    reg [3:0] Q_reg;  
    reg Q_minus1;  
  
    integer i;  
  
    always @(*) begin  
  
        // Initialization  
        A = 5'b00000;  
        M_ext = {M[3], M};  
        Q_reg = Q;  
        Q_minus1 = 1'b0;  
  
        // Four Booth iterations  
        for (i = 0; i < 4; i = i + 1) begin  
  
            // Booth operation  
            case ({Q_reg[0], Q_minus1})  
  
                2'b01: A = A + M_ext;  
                2'b10: A = A - M_ext;  
                2'b00: A = A;  
                2'b11: A = A;  
  
            endcase  
  
            // Arithmetic right shift  
            Q_minus1 = Q_reg[0];  
            Q_reg = {A[0], Q_reg[3:1]};  
            A = {A[4], A[4:1]};  
  
        end  
  
        // Final product
```

```
Product = {A[3:0], Q_reg};
```

```
Endmodule
```

Using Testbench:

```
`timescale 1ns/1ps
```

```
module multiplier_tb;
```

```
reg [3:0] A, B;
```

```
reg signed [3:0] M, Q;
```

```
wire [7:0] array_product;
```

```
wire signed [7:0] booth_product;
```

```
array_multiplier_4x4 AM (  
    .A(A),  
    .B(B),  
    .Product(array_product)  
);
```

```
booth_multiplier_4x4 BM (  
    .M(M),  
    .Q(Q),  
    .Product(booth_product)  
);
```

```
initial begin
```

```
    // Test 1
```

```
A = 4'd3;
```

```
B = 4'd5;
```

```
M = 4'sd3;
```

```
Q = 4'sd5;
```

```
#10;
```

```
    // Test 2
```

```
A = 4'd7;
```

```
B = 4'd8;
```

```
M = -4'sd3;
```

```
Q = 4'sd5;
```

```
#10;
```

```
    // Test 3
```

```
A = 4'd10;
```

```
B = 4'd6;  
M = 4'sd4;  
Q = -4'sd2;  
#10;
```

```
// Test 4  
A = 4'd15;  
B = 4'd15;  
M = -4'sd4;  
Q = -4'sd2;  
#10;
```

```
$finish;
```

```
end
```

```
Endmodule
```

Verification Testbench

- A common testbench applies multiple input combinations.
- Array multiplier test cases:
 - 3×5
 - 7×8
 - 10×6
 - 15×15
- Booth multiplier test cases:
 - 3×5
 - -3×5
 - $4 \times (-2)$
 - $-4 \times (-2)$
 - 7×7
- Expected results are checked against the simulated outputs.
- Waveforms are used to observe **inputs and products** over simulation time.

Simulation Results

Array Multiplier:

- $3 \times 5 = 15$
- $7 \times 8 = 56$
- $10 \times 6 = 60$
- $15 \times 15 = 225$

Booth Multiplier:

- $3 \times 5 = 15$
- $-3 \times 5 = -15$
- $4 \times (-2) = -8$
- $-4 \times (-2) = 8$
- $7 \times 7 = 49$

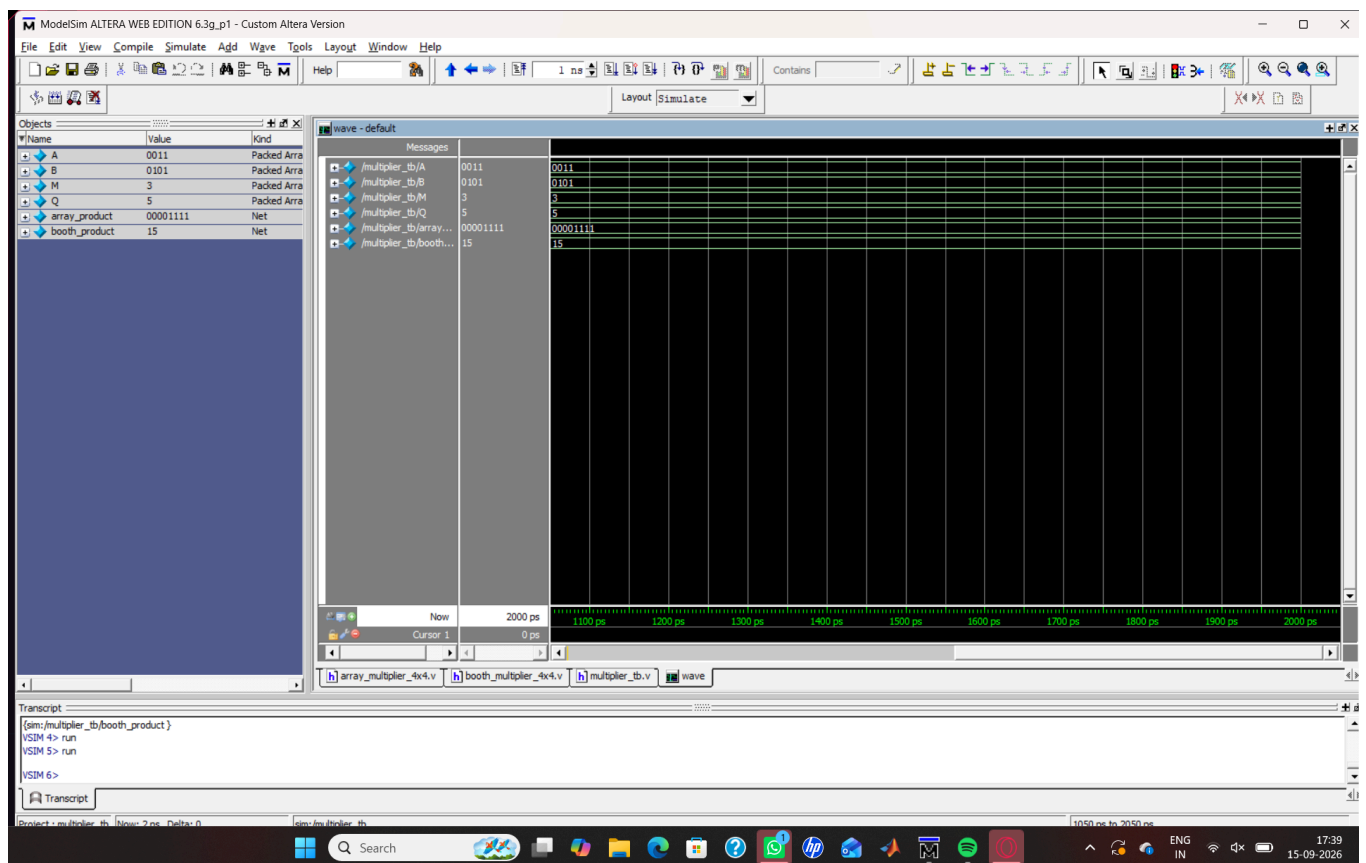
These cases cover **positive and signed negative operands**.

Waveform Explanation

- At each test interval, the testbench changes the **input operands**.
- The corresponding product changes after **combinational logic settles**.
- For the array multiplier, observe:
 - A
 - B
 - array_product

For the Booth multiplier, observe:

- M
- Q
- booth_product
 -



Array vs Booth Multiplier

Array Multiplier:

- Simple and regular structure.
- Generates all partial products.

Booth Multiplier:

- Uses add/subtract decisions.
- Uses arithmetic shifting.
- Naturally supports signed two's-complement multiplication.
- Can reduce arithmetic operations for suitable multiplier patterns

Synthesis and VLSI Analysis

- Synthesis converts **RTL into a gate-level implementation.**
- Useful results include:
 - Cell/gate count
 - Area
 - Timing

- Maximum frequency or critical-path delay can also be reported.
- Power can be analyzed when supported by the synthesis flow.
- **Actual tool-generated values should be used; no results are assumed in this project.**

Conclusion

- **Both 4×4 multiplier architectures can produce the required products correctly.**
- **The array multiplier provides a simple and regular implementation.**
- **The Booth multiplier provides signed multiplication and can reduce arithmetic operations.**
- **Simulation verifies functional correctness for positive and negative operands.**
- **The project demonstrates:**
 - **RTL design**
 - **Verification**
 - **Waveform analysis**
 - **Architectural comparison**

THANK YOU